National University of Computer and Emerging Sciences

Lab Manual 10

Department of Data Science
FAST- NU, Lahore, Pakistan

Single Perceptron

A perceptron is a neural network unit (an artificial neuron) that does certain computations to detect features or business intelligence in the input data. A Perceptron is an algorithm for supervised learning of binary classifiers. This algorithm enables neurons to learn and process elements in the training set one at a time.

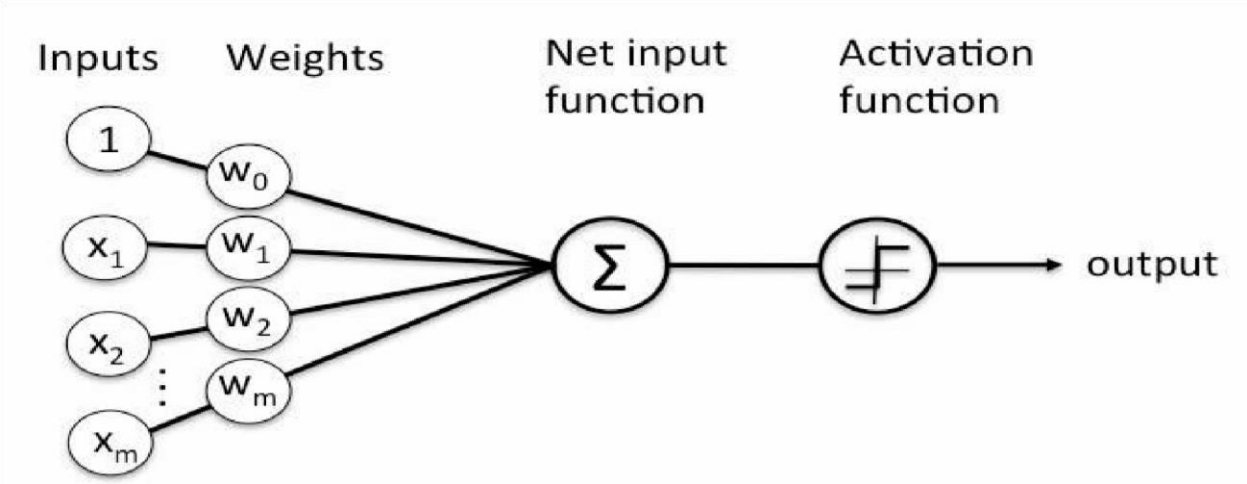

Figure 1 Single Perceptron Illustration

1.1 Types of Perceptrons

Single layer Perceptrons can learn only linearly separable patterns.

Multilayer Perceptrons or feedforward neural networks with two or more layers have the greater processing power.

1.2 Perceptron Function

Perceptron is a function that maps its input "x," which is multiplied with the learned weight coefficient; an output value "f(x)"is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

In the equation given above:

"w" = vector of real-valued weights

"b" = bias (an element that adjusts the boundary away from origin without any dependence on the input value)
"x" = vector of input x values

$$\sum_{i=1}^{m} w_i x_i$$

"m" = number of inputs to the Perceptron

The output can be represented as "1" or "0." It can also be represented as "1" or "-1" depending on which activation function is used.

1.3 Inputs of a Perceptron

A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result. The image below shows a Perceptron with a Boolean output.

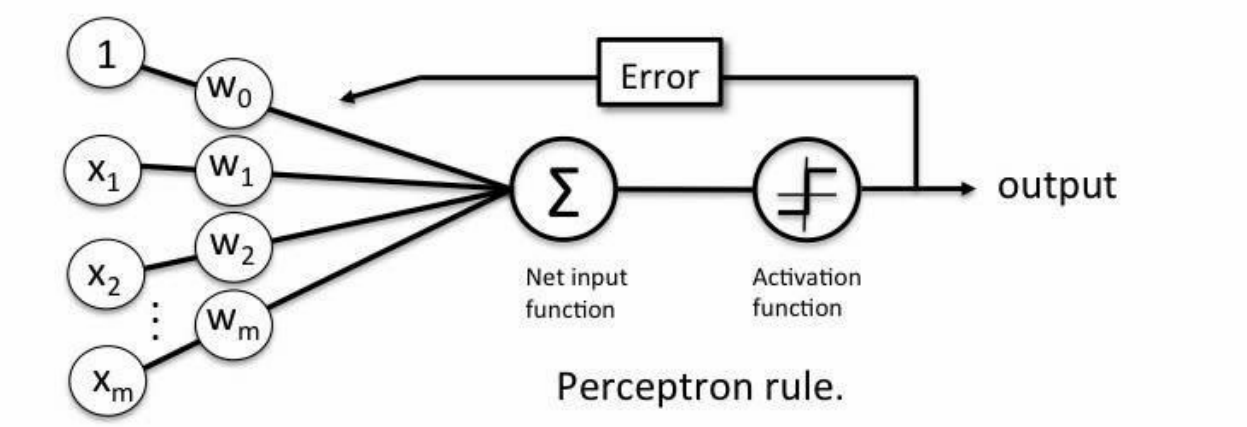

Figure 2 Inputs in a Perceptron Learning Process

A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False. The summation function "∑" multiplies all inputs of "x" by weights "w" and then adds them up as follows:

$$w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

1.4 Activation Functions of Perceptron

The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.

For example:

```
If ∑ wixi> 0 => then final output "o" = 1 (issue bank loan)
Else, final output "o" = -1 (deny bank loan)
```

Step function gets triggered above a certain value of the neuron output; else it outputs zero.

1.5 Perceptron at a glance

- Weights are multiplied with the input features and decision is made if the neuron is fired or not.
- Activation function applies a step rule to check if the output of the weighting function is greater than zero.
- Linear decision boundary is drawn enabling the distinction between the two linearly separable classes +1 and -1.
- If the sum of the input signals exceeds a certain threshold, it outputs a signal; otherwise, there is no output.
- Types of activation functions include the sign, step, and sigmoid functions.

1.6 Perceptron Training Algorithms

In this course, we have covered training algorithms namely, perceptron learning.

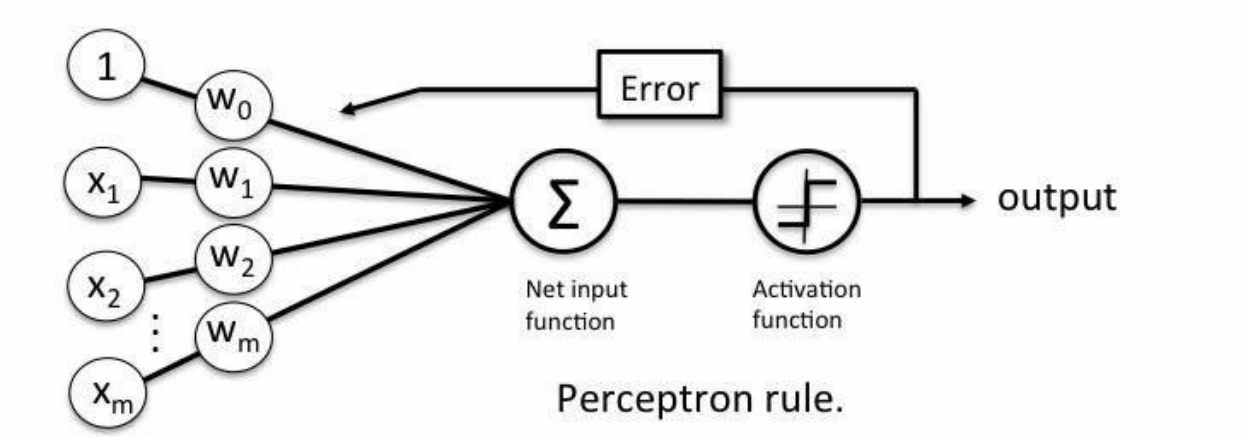

Figure 4 Learning Process

The perceptron receives multiple input signals and if the sum of the input signals exceeds a certain threshold, it either outputs a signal or does not return an output.

In this lab session, we only focus on the perceptron learning rule. This rule requires the data to be linearly separable. The weight update rule is given by:

$$w_i = w_i + \Delta w_i$$

where

$$\Delta w_i = \eta \cdot (y - \hat{y}) \cdot x_i$$

and

$$b = b + \eta \cdot (y - \hat{y})$$

Where:

- y is the actual output,
- $\hat{y}$ is the perceptron's predicted output,
- η is the learning rate (a small constant that controls how much weights change).

1.7 Logic Gates

Logic gates are the building blocks of a digital system, especially neural network. In short, they are the electronic circuits that help in addition, choice, negation, and combination to form complex circuits.

Using the logic gates, Neural Networks can learn on their own without you having to manually code the logic. Most logic gates have two inputs and one output.

Each terminal has one of the two binary conditions, low (0) or high (1), represented by different voltage levels. The logic state of a terminal changes based on how the circuit processes data. Based on this logic, logic gates can be categorized into seven types:

- AND
- NAND
- OR
- NOR • NOT • XOR

Introduction to tensorflow and pytorch:

Modern Artificial Intelligence (AI) and Machine Learning (ML) applications rely heavily on *deep learning* frameworks that simplify the implementation and training of complex neural networks. Two of the most widely used open-source libraries for this purpose are **TensorFlow** and **PyTorch**. Both are built for **numerical computation using dataflow graphs** and provide powerful APIs for building Artificial Neural Networks (ANNs) efficiently.

What is TensorFlow?

TensorFlow is an open-source platform for machine learning and deep learning developed by Google. It allows researchers and developers to build and train neural networks using multi-dimensional arrays called **tensors**.

Key Features of TensorFlow

- **Tensors:** Core data structure; represents data as n-dimensional arrays.
- **Computation Graphs:** Operations are represented as nodes in a graph (static or eager execution mode).
- **Keras API:** High-level interface in TensorFlow that simplifies model creation, training, and evaluation.
- **GPU/TPU Support:** TensorFlow can leverage hardware acceleration for faster training.
- **Model Deployment:** Integration with TensorFlow Lite, TensorFlow.js, and TensorFlow Serving allows models to be deployed on mobile, web, and production servers.

What is PyTorch?

PyTorch is an open-source deep learning framework developed by Facebook (Meta) that provides a flexible and dynamic way to define and train neural networks. Unlike TensorFlow's early versions, which used static computation graphs, PyTorch uses **dynamic computation graphs**, allowing changes in network structure during runtime — very useful for research and experimentation.

Key Features of PyTorch

- **Dynamic Computation Graphs:** Build and modify networks on the fly during execution.
- **Pythonic Syntax:** Seamless integration with Python libraries such as NumPy.
- **Automatic Differentiation:** The `autograd` module computes gradients automatically for backpropagation.
- **TorchScript & Deployment:** Models can be exported for production with TorchScript or deployed via ONNX.
- **GPU Support:** Built-in CUDA support for efficient GPU computations.

Lab Tasks

1 Implement a **single-layer perceptron** to model the **AND** logic gate.

x_1	x_2	y_{AND}
0	0	0
0	1	0
1	0	0
1	1	1

Do not use built in functions.

```
# Step function (Activation function)
def step_function(z)

# Perceptron Learning Algorithm
def train_perceptron(X, y, epochs=10, learning_rate=0.1)

# Predict function
def predict(X, weights, bias)

#Plot decision boundary
def plot_decision_boundary(X, y, weights, bias, title)
```

You can get help here: https://medium.com/analytics-vidhya/implementing-perceptron-learning-algorithm-to-solve-and-in-python-903516300b2f

2. Implementation of a Single-Layer Perceptron using TensorFlow

1. Go through the following tutorial carefully:
   🔗 Single-Layer Perceptron in TensorFlow – GeeksforGeeks
2. Follow all the steps given in the tutorial to:
   - Understand the concept of a single-layer perceptron.
   - Implement the perceptron using TensorFlow and Python.
   - Train and test the model on the sample dataset provided in the tutorial.
3. Observe and note the following:

- The architecture of the perceptron (number of inputs, activation function, output).
- Training parameters such as optimizer, loss function, and number of epochs.
- Final accuracy or performance obtained.